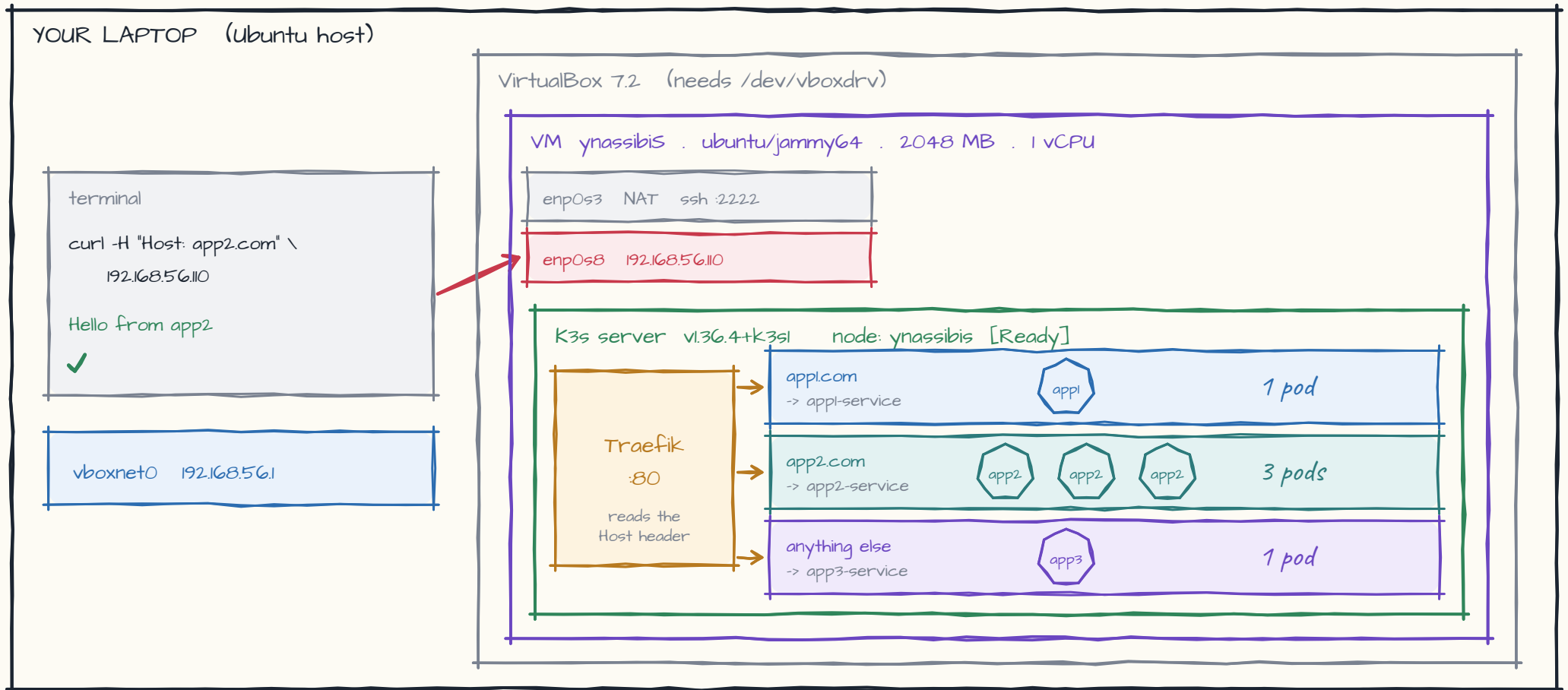


p2 on one page

One VM. One node. One IP. Everything the evaluator touches lives behind 192.168.56.110:80



5 pods, 3 services, 1 ingress, 1 node -- and from the outside it is just one IP on port 80.

Three files build the whole thing

Vagrantfile makes the box. setup.sh installs the cluster. confs/ describes the apps.

Vagrantfile

- box = ubuntu/jammy64
- private_network -> .10 (2nd NIC)
- synced_folder type: rsync
- provision: scripts/setup.sh
- vb.name = ynassibis

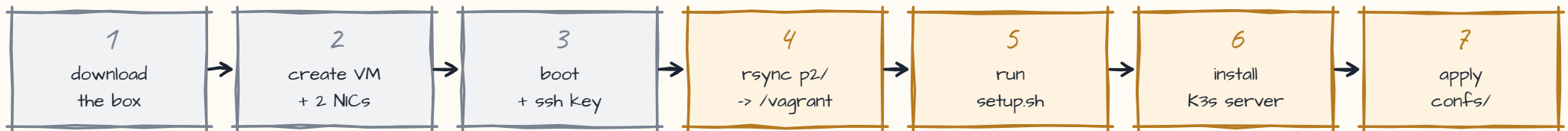
scripts/setup.sh

- find the NIC that owns .10
- install K3s in SERVER mode
- wait until the node is Ready
- kubectl apply -f /vagrant/confs/
- copy kubeconfig to vagrant user

confs/*.yaml

- app1 / app2 / app3
- each = Deployment + Service
- ingress.yaml routes by Host
- pure desired state -
- no scripting, no order

what 'vagrant up' actually does, in order:



'vagrant provision' re-runs only steps 4 - 7

so setup.sh must be idempotent

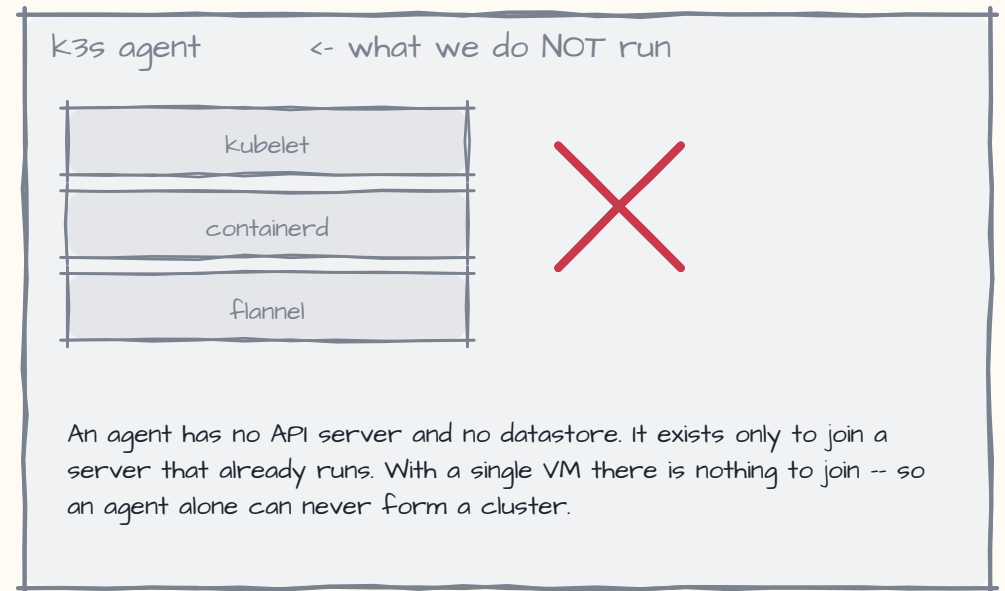
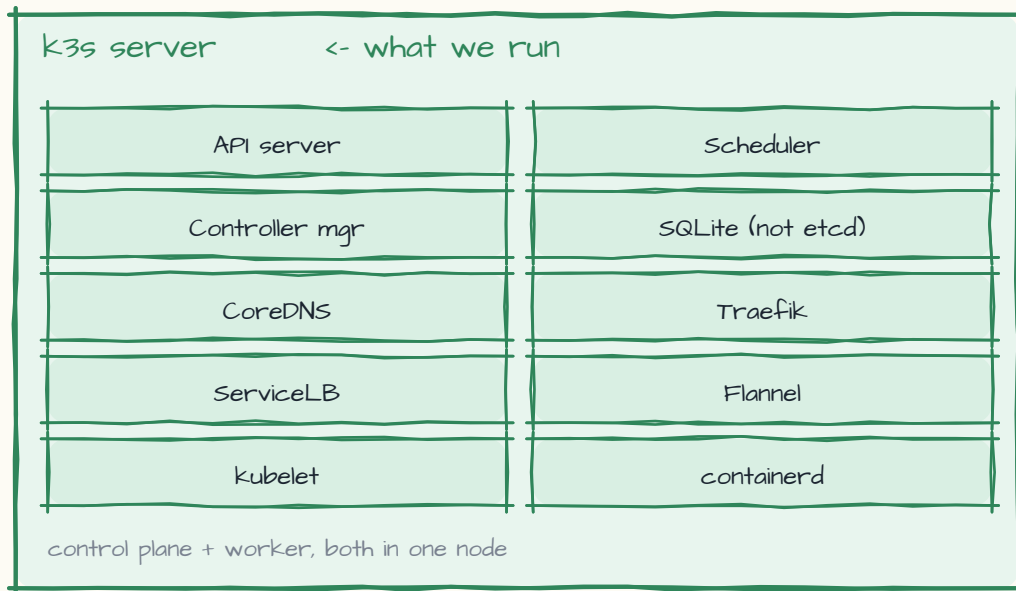
It is re-run whole, every time. The K3s installer and kubectl apply are both safe to repeat; the bashrc line is guarded with grep -q.

rsync is a copy, not a live mount

Editing confs/ on the host changes nothing inside the VM until the next rsync. ubuntu/jammy64 ships no Guest Additions, so vboxsf cannot mount at all.

Why SERVER mode?

A Kubernetes cluster needs a brain. With exactly one VM, that VM has to be the brain.



```
server --node-ip=192.168.56.110 --flannel-iface=enp0s8 --write-kubeconfig-mode=644
```

--node-ip

pins the node's internal IP to .110. Without it Kubernetes picks the NAT address and the evaluator's curl finds nothing.

--flannel-iface

the pod network must ride the private NIC. setup.sh resolves it at runtime (enp0s8 here) -- never hardcode eth1.

--write-kubeconfig-mode

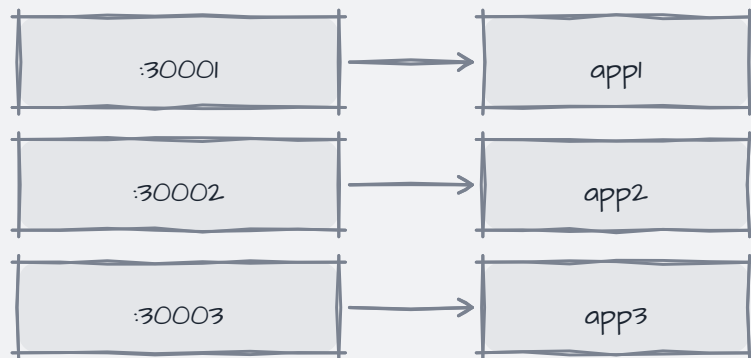
makes /etc/rancher/k3s/k3s.yaml world-readable so the vagrant user runs kubectl without sudo.

Why an Ingress, not three NodePorts?

The subject asks for `app1.com` and `app2.com` on one address. Only an Ingress can do that.

WITHOUT an Ingress

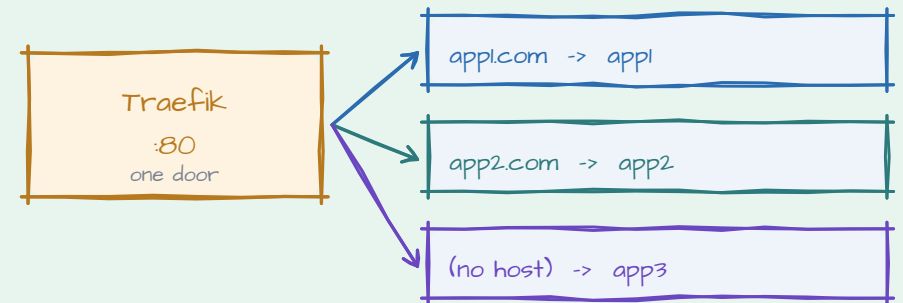
three services, three random ports



- ✗ does not match the subject
- ✗ evaluator must know 3 ports
- ✗ no host names at all

WITH an Ingress

one port, routed by the Host header



- ✓ `app3` is the default catch-all
- ✓ one IP, one port, three names
- ✓ exactly what the subject asks

`ingress.yaml` -- read it top to bottom:

`host: app1.com`

→ `app1-service:80`

matched on the HTTP Host header

`host: app2.com`

→ `app2-service:80`

matched on the HTTP Host header

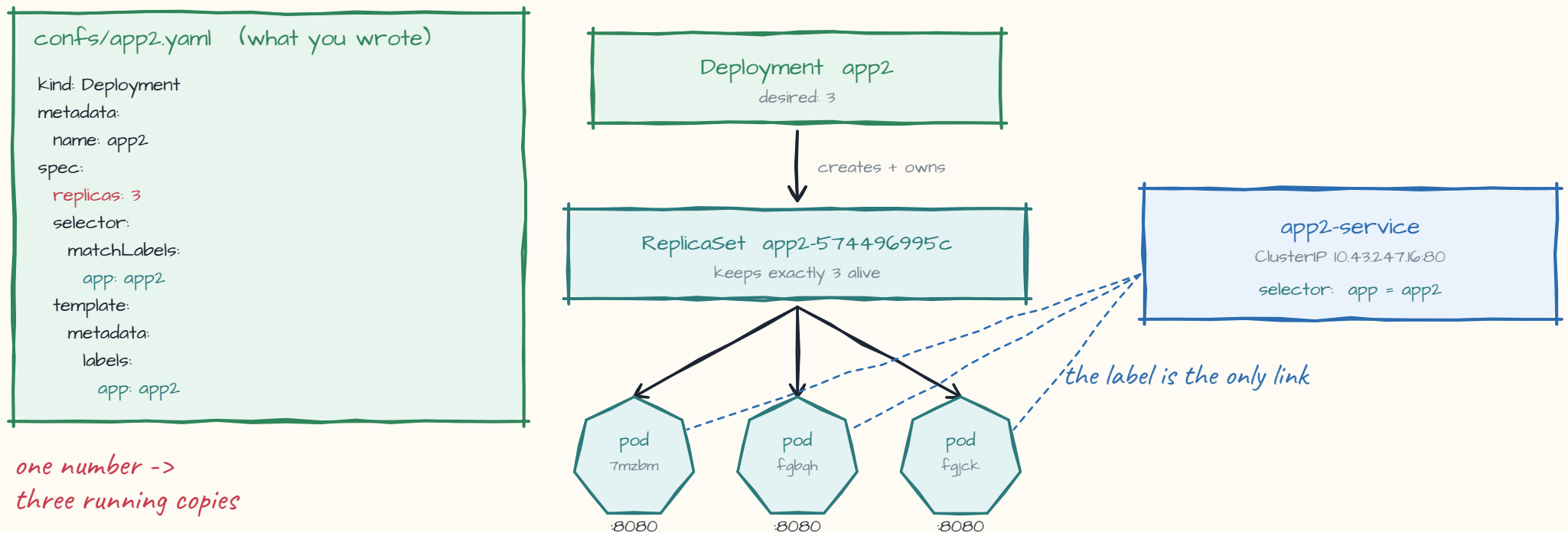
(no host: field)

→ `app3-service:80`

the 'otherwise' case in the subject

How 'replicas: 3' becomes 3 pods

You never create a pod. You declare a Deployment, and two controllers do the rest.



one number ->
three running copies

the control loop, forever

Kill a pod and the ReplicaSet notices the count dropped to 2 and starts a third. Change replicas to 5 and the Deployment makes a new ReplicaSet decision. Nothing about this is scripted -- it is a loop comparing desired state to real state.

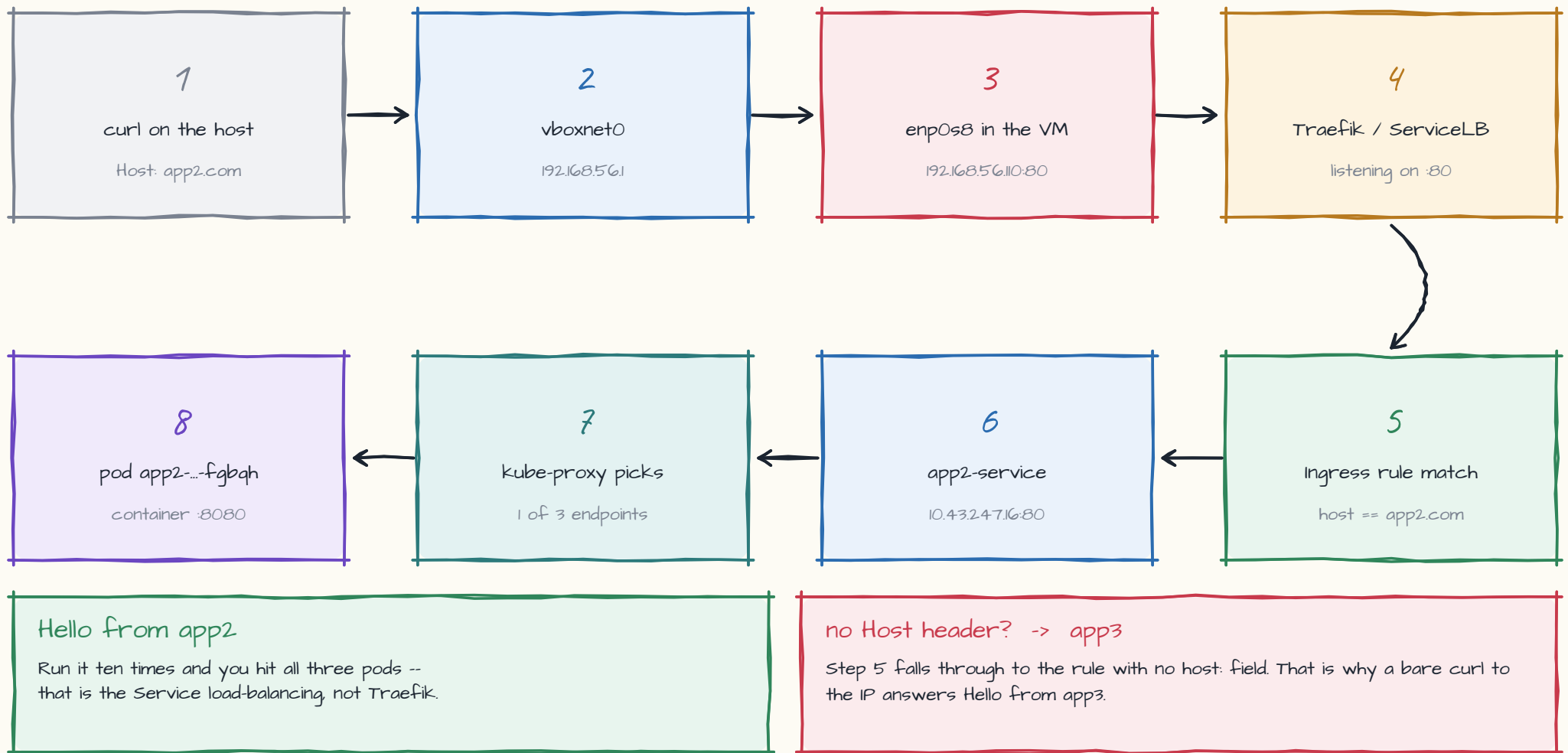
app1 and app3 are the same shape

Identical YAML with replicas: 1 and a different MESSAGE env var. All three run the same image, paulbouwer/hello-kubernetes:1.0, listening on 8080 -- the Service maps port 80 to targetPort 8080.

One request, end to end

`curl -H "Host: app2.com" 192.168.56.110 -- eight hops to "Hello from app2"`

the packet leaves your shell and never touches DNS -- the Host header is just a string in the request



Things that bite (and why)

Six failures worth recognising before the evaluation, not during it.

rsync is not a live mount

ubuntu/jammy64 ships without Guest Additions, so vboxsf cannot mount /vagrant at all. Edits to `confs/` reach the VM only on the next 'vagrant rsync' or 'vagrant provision'.

the second NIC is the real one

`enp0s3` is NAT and identical on every VM. The .110 address lives on `enp0s8`. `setup.sh` finds it by IP at runtime -- hardcoding `eth1` breaks on Jammy, where names are `enp0sN`.

192.168.56.0/24 or nothing

VirtualBox refuses any other host-only range unless you list it in `/etc/vbox/networks.conf`. Pick .57.x on a whim and 'vagrant up' fails creating the interface.

Secure Boot vs vboxdrv

DKMS signs the modules with a key your firmware does not trust, so `/dev/vboxdrv` never appears and every 'vagrant up' dies. Enroll the MOK key, or turn Secure Boot off.

setup.sh runs again, whole

'vagrant provision' re-runs it from the top. The K3s installer and `kubectl apply` are safe to repeat; the `KUBECONFIG` line in `.bashrc` is guarded with `grep -q`.

the node name is lowercase

The VM hostname is `ynassibis` but 'kubectl get nodes' shows `ynassibis`. Kubernetes lowercases object names. Nothing is broken -- do not panic at evaluation.

If curl returns nothing at all, the fault is usually below Kubernetes.

Check `vboxnet0` is UP on the host, then `/dev/vboxdrv`, then `enp0s8` inside the VM -- in that order.